Contents

Data Structures Cheat Sheet **1**
Tabla de Contenidos . . . 1
1. Resumen de Estructuras . . . 1
2. Listas (list) . . . 2
3. Tuplas (tuple) . . . 4
4. Diccionarios (dict) . . . 6
5. Sets (set, frozenset) . . . 9
6. Estructuras Anidadas . . . 11
7. Comprensiones . . . 12
8. Funciones Útiles . . . 14
9. Patrones Comunes . . . 16
10. Rendimiento (Big O) . . . 18
Recursos Relacionados . . . 19

Data Structures Cheat Sheet

Python 3.13+ | Listas, Tuplas, Diccionarios, Sets

Tabla de Contenidos

1. Resumen de Estructuras
2. Listas (list)
3. Tuplas (tuple)
4. Diccionarios (dict)
5. Sets (set, frozenset)
6. Estructuras Anidadas
7. Comprensiones
8. Funciones Útiles
9. Patrones Comunes
10. Rendimiento (Big O)

---

1. Resumen de Estructuras

Estructura	Ordenada	Mutable	Duplicados	Indexable	Sintaxis
`list`					`[1, 2, 3]`
`tuple`					`(1, 2, 3)`
`dict`	*		Keys:	Por key	`{"a": 1}`
`set`					`{1, 2, 3}`
`frozenset`					`frozenset({1, 2})`

** Ordenados por inserción desde Python 3.7+*

Cuándo Usar Cada Uno

Necesidad	Estructura
Colección ordenada modificable	`list`
Datos inmutables, keys de dict	`tuple`
Mapeo key→value	`dict`
Elementos únicos, operaciones de conjuntos	`set`

Necesidad	Estructura
Set inmutable, key de dict	`frozenset`

2. Listas (list)

Creación

```
# Literal
numbers = [1, 2, 3, 4, 5]
mixed = [1, "two", 3.0, True]
empty = []

# Constructor
list("abc")           # ['a', 'b', 'c']
list(range(5))        # [0, 1, 2, 3, 4]
list((1, 2, 3))       # [1, 2, 3]

# Comprensión
squares = [x**2 for x in range(10)]

# Repetición
zeros = [0] * 5       # [0, 0, 0, 0, 0]

#  Cuidado con listas anidadas
matrix = [[0] * 3] * 3    #  Misma referencia!
matrix = [[0] * 3 for _ in range(3)]  #  Diferente
```

Acceso e Indexación

```
nums = [10, 20, 30, 40, 50]
#       0   1   2   3   4    (índices positivos)
#      -5  -4  -3  -2  -1    (índices negativos)

# Acceso por índice
nums[0]      # 10 (primero)
nums[-1]     # 50 (último)
nums[2]      # 30

# Slicing [start:stop:step]
nums[1:4]    # [20, 30, 40]
nums[:3]     # [10, 20, 30]
nums[2:]     # [30, 40, 50]
nums[::2]    # [10, 30, 50] (cada 2)
nums[::-1]   # [50, 40, 30, 20, 10] (reverso)
nums[1:-1]   # [20, 30, 40] (sin primero ni último)

# Asignación por slice
nums[1:3] = [200, 300]     # [10, 200, 300, 40, 50]
nums[1:3] = [200]          # [10, 200, 40, 50] (reduce)
nums[1:1] = [15, 16]       # Insertar sin eliminar
```

Métodos de Lista

```python
lst = [1, 2, 3]

lst.append(4)         # [1, 2, 3, 4] - Agregar al final
lst.insert(0, 0)      # [0, 1, 2, 3, 4] - Insertar en índice
lst.extend([5, 6])    # [0, 1, 2, 3, 4, 5, 6] - Agregar múltiples

# Concatenación (crea nueva lista)
new_lst = lst + [7, 8]
lst += [7, 8]         # Equivale a extend
```

Agregar Elementos

```python
lst = [1, 2, 3, 2, 4]

lst.remove(2)         # [1, 3, 2, 4] - Primera ocurrencia
item = lst.pop()      # Elimina y retorna último (4)
item = lst.pop(0)     # Elimina y retorna índice 0 (1)
lst.clear()           # [] - Vaciar lista

del lst[0]            # Eliminar por índice
del lst[1:3]          # Eliminar por slice
```

Eliminar Elementos

```python
lst = [10, 20, 30, 20, 40]

lst.index(20)         # 1 (primer índice)
lst.index(20, 2)      # 3 (buscar desde índice 2)
lst.count(20)         # 2 (ocurrencias)

20 in lst             # True
50 not in lst         # True

# Búsqueda segura
try:
    idx = lst.index(99)
except ValueError:
    idx = -1
```

Búsqueda

```python
nums = [3, 1, 4, 1, 5, 9]

# In-place (modifica original)
nums.sort()                     # [1, 1, 3, 4, 5, 9]
nums.sort(reverse=True)         # [9, 5, 4, 3, 1, 1]
nums.sort(key=lambda x: -x)     # Orden descendente

# Nueva lista ordenada
sorted_nums = sorted(nums)
sorted_nums = sorted(nums, reverse=True)

# Ordenar por criterio
```

```
words = ["banana", "Apple", "cherry"]
words.sort(key=str.lower)      # Case-insensitive
words.sort(key=len)            # Por longitud

# Ordenar objetos
users = [{"name": "Bob", "age": 30}, {"name": "Alice", "age": 25}]
users.sort(key=lambda u: u["age"])

# Múltiples criterios
users.sort(key=lambda u: (u["age"], u["name"]))
```

Ordenación

```
lst = [1, 2, 3]

lst.reverse()         # [3, 2, 1] - In-place
reversed_lst = list(reversed(lst))  # Nueva lista

lst.copy()            # Copia superficial
import copy
deep = copy.deepcopy(nested_list)   # Copia profunda
```

Otros Métodos

Operaciones de Lista

```
# Longitud
len([1, 2, 3])        # 3

# Min, Max, Sum
min([3, 1, 4])        # 1
max([3, 1, 4])        # 4
sum([1, 2, 3])        # 6

# Verificar todos/alguno
all([True, True, False])   # False
any([False, False, True])  # True

# Contar elementos
from collections import Counter
Counter([1, 1, 2, 3, 3, 3])  # Counter({3: 3, 1: 2, 2: 1})
```

---

3. Tuplas (tuple)

Creación

```
# Literal
point = (10, 20)
single = (42,)        #  Coma necesaria para un elemento
empty = ()

# Sin paréntesis
coords = 10, 20, 30
a, b, c = coords      # Desempaquetado
```

```
# Constructor
tuple([1, 2, 3])      # (1, 2, 3)
tuple("abc")          # ('a', 'b', 'c')
```

Características

```
# Inmutable - NO se puede modificar
t = (1, 2, 3)
t[0] = 10             #  TypeError

# Pero elementos mutables sí
t = ([1, 2], [3, 4])
t[0].append(3)        #  ([1, 2, 3], [3, 4])

# Hashable (si todos los elementos son hashables)
{(1, 2): "point"}     #  Puede ser key de dict
{([1, 2],): "list"}   #  Lista no es hashable
```

Operaciones

```
t = (1, 2, 3, 2, 4)

# Acceso e indexación (igual que listas)
t[0]                  # 1
t[-1]                 # 4
t[1:3]                # (2, 3)

# Métodos
t.count(2)            # 2
t.index(3)            # 2

# Concatenación y repetición (crea nueva tupla)
t + (5, 6)            # (1, 2, 3, 2, 4, 5, 6)
(1, 2) * 3            # (1, 2, 1, 2, 1, 2)

# Desempaquetado
x, y, z, *rest = (1, 2, 3, 4, 5)
# x=1, y=2, z=3, rest=[4, 5]
```

Named Tuples

```
from collections import namedtuple

# Definir tipo
Point = namedtuple('Point', ['x', 'y'])
User = namedtuple('User', 'name age email')

# Crear instancias
p = Point(10, 20)
u = User("Alice", 30, "alice@example.com")

# Acceso
p.x                   # 10
p[0]                  # 10
```

```
u.name                # "Alice"

# Conversión
p._asdict()           # {'x': 10, 'y': 20}
Point._make([10, 20]) # Point(x=10, y=20)

# Crear copia modificada
p._replace(x=100)     # Point(x=100, y=20)
```

Typing con NamedTuple

```
from typing import NamedTuple

class Point(NamedTuple):
    x: float
    y: float
    label: str = "origin"  # Valor por defecto

p = Point(10.5, 20.3)
p = Point(10.5, 20.3, "A")
```

---

4. Diccionarios (dict)

Creación

```
# Literal
person = {"name": "Alice", "age": 30}
empty = {}

# Constructor
dict(name="Alice", age=30)
dict([("name", "Alice"), ("age", 30)])
dict.fromkeys(["a", "b", "c"], 0)  # {"a": 0, "b": 0, "c": 0}

# Comprensión
squares = {x: x**2 for x in range(5)}
```

Acceso y Modificación

```
d = {"name": "Alice", "age": 30}

# Acceso
d["name"]             # "Alice"
d["unknown"]          #  KeyError

# Acceso seguro
d.get("name")         # "Alice"
d.get("unknown")      # None
d.get("unknown", "N/A")  # "N/A" (valor por defecto)

# Modificación
d["age"] = 31         # Actualizar
d["city"] = "NYC"     # Agregar nuevo
```

```python
# Actualizar múltiples
d.update({"age": 32, "country": "USA"})
d |= {"age": 33}      # Python 3.9+

# Eliminar
del d["city"]
age = d.pop("age")              # Elimina y retorna
item = d.popitem()              # Elimina y retorna último (LIFO)
d.clear()                       # Vaciar
```

Métodos de Diccionario

```python
d = {"a": 1, "b": 2, "c": 3}

# Vistas (actualizadas dinámicamente)
d.keys()              # dict_keys(['a', 'b', 'c'])
d.values()            # dict_values([1, 2, 3])
d.items()             # dict_items([('a', 1), ('b', 2), ('c', 3)])

# Convertir a lista
list(d.keys())        # ['a', 'b', 'c']

# setdefault - obtener o establecer
d.setdefault("d", 4)  # Retorna 4, agrega si no existe
d.setdefault("a", 10) # Retorna 1 (ya existe, no cambia)

# Copiar
shallow = d.copy()
import copy
deep = copy.deepcopy(nested_dict)
```

Iteración

```python
d = {"a": 1, "b": 2, "c": 3}

# Iterar keys (por defecto)
for key in d:
    print(key)

# Iterar values
for value in d.values():
    print(value)

# Iterar items (key, value)
for key, value in d.items():
    print(f"{key}: {value}")

# Con índice
for i, (key, value) in enumerate(d.items()):
    print(f"{i}: {key}={value}")
```

Merge de Diccionarios

```python
d1 = {"a": 1, "b": 2}
d2 = {"b": 3, "c": 4}
```

```python
# Python 3.9+ - Operador |
merged = d1 | d2      # {"a": 1, "b": 3, "c": 4}
d1 |= d2              # In-place merge

# Método tradicional
merged = {**d1, **d2}
merged = d1.copy()
merged.update(d2)

# ChainMap (vista sin copiar)
from collections import ChainMap
combined = ChainMap(d2, d1)  # d2 tiene prioridad
```

defaultdict

```python
from collections import defaultdict

# Lista por defecto
groups = defaultdict(list)
groups["fruits"].append("apple")
groups["fruits"].append("banana")
# defaultdict(list, {'fruits': ['apple', 'banana']})

# Entero por defecto (contador)
counts = defaultdict(int)
for word in words:
    counts[word] += 1

# Set por defecto
unique_items = defaultdict(set)

# Función personalizada
def default_user():
    return {"name": "Unknown", "active": False}

users = defaultdict(default_user)
```

Counter

```python
from collections import Counter

# Contar elementos
c = Counter("abracadabra")
# Counter({'a': 5, 'b': 2, 'r': 2, 'c': 1, 'd': 1})

c = Counter([1, 1, 2, 3, 3, 3])
# Counter({3: 3, 1: 2, 2: 1})

# Operaciones
c.most_common(2)      # [(3, 3), (1, 2)]
c["a"]                # 5
c["z"]                # 0 (no KeyError)

# Actualizar
c.update("aaa")       # Agregar más conteos
```

```
c.subtract("aa")      # Restar conteos

# Operaciones de conjuntos
c1 + c2               # Sumar conteos
c1 - c2               # Restar (sin negativos)
c1 & c2               # Mínimo de cada
c1 | c2               # Máximo de cada
```

---

5. Sets (set, frozenset)

Creación

```
# Literal
numbers = {1, 2, 3}
empty = set()         #  {} crea dict vacío!

# Constructor
set([1, 2, 2, 3])     # {1, 2, 3} - Elimina duplicados
set("hello")          # {'h', 'e', 'l', 'o'}

# Comprensión
evens = {x for x in range(10) if x % 2 == 0}

# frozenset (inmutable)
fs = frozenset([1, 2, 3])
```

Agregar y Eliminar

```
s = {1, 2, 3}

# Agregar
s.add(4)              # {1, 2, 3, 4}
s.update([5, 6])      # {1, 2, 3, 4, 5, 6}

# Eliminar
s.remove(6)           # Elimina, error si no existe
s.discard(99)         # Elimina, sin error si no existe
item = s.pop()        # Elimina y retorna arbitrario
s.clear()             # Vaciar
```

Operaciones de Conjuntos

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

# Unión (todos los elementos)
a | b                 # {1, 2, 3, 4, 5, 6}
a.union(b)

# Intersección (elementos comunes)
a & b                 # {3, 4}
a.intersection(b)

# Diferencia (en a pero no en b)
```

```
a - b                   # {1, 2}
a.difference(b)

# Diferencia simétrica (en uno pero no en ambos)
a ^ b                   # {1, 2, 5, 6}
a.symmetric_difference(b)

# Operaciones in-place
a |= b                  # a.update(b)
a &= b                  # a.intersection_update(b)
a -= b                  # a.difference_update(b)
a ^= b                  # a.symmetric_difference_update(b)
```

Comparaciones de Conjuntos

```
a = {1, 2, 3}
b = {1, 2}
c = {1, 2, 3}

# Subconjunto
b <= a                  # True (b es subconjunto de a)
b < a                   # True (subconjunto propio)
b.issubset(a)

# Superconjunto
a >= b                  # True
a > b                   # True
a.issuperset(b)

# Igualdad
a == c                  # True

# Disjuntos (sin elementos comunes)
{1, 2}.isdisjoint({3, 4})  # True
```

Casos de Uso

```
# Eliminar duplicados preservando orden
def unique_ordered(items):
    seen = set()
    return [x for x in items if not (x in seen or seen.add(x))]

# Verificación rápida de membresía
valid_users = {"alice", "bob", "charlie"}
if username in valid_users:
    allow_access()

# Encontrar elementos únicos
unique = set(list1) - set(list2)

# Verificar elementos comunes
has_common = bool(set(list1) & set(list2))
```

---

6. Estructuras Anidadas

Listas Anidadas (Matrices)

```
# Crear matriz 3x3
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

# Acceso
matrix[1][2]          # 6 (fila 1, columna 2)

# Iterar
for row in matrix:
    for cell in row:
        print(cell)

# Comprensión para crear
matrix = [[0] * 3 for _ in range(3)]

# Comprensión para transformar
doubled = [[cell * 2 for cell in row] for row in matrix]

# Transponer
transposed = [[row[i] for row in matrix] for i in range(len(matrix[0]))]
# O con zip
transposed = list(map(list, zip(*matrix)))

# Aplanar
flat = [cell for row in matrix for cell in row]
# O
import itertools
flat = list(itertools.chain.from_iterable(matrix))
```

Diccionarios Anidados

```
users = {
    "alice": {
        "age": 30,
        "email": "alice@example.com",
        "roles": ["admin", "user"]
    },
    "bob": {
        "age": 25,
        "email": "bob@example.com",
        "roles": ["user"]
    }
}

# Acceso
users["alice"]["email"]          # "alice@example.com"
users["alice"]["roles"][0]       # "admin"

# Acceso seguro anidado
```

```python
def get_nested(d, *keys, default=None):
    for key in keys:
        if isinstance(d, dict):
            d = d.get(key, default)
        else:
            return default
    return d

get_nested(users, "alice", "email")        # "alice@example.com"
get_nested(users, "charlie", "email")      # None
get_nested(users, "alice", "phone", default="N/A")  # "N/A"

# Modificar
users["alice"]["age"] = 31
users["alice"]["roles"].append("moderator")

# Agregar nuevo usuario
users["charlie"] = {"age": 35, "email": "charlie@example.com", "roles": []}
```

Listas de Diccionarios

```python
employees = [
    {"name": "Alice", "dept": "Engineering", "salary": 75000},
    {"name": "Bob", "dept": "Marketing", "salary": 65000},
    {"name": "Charlie", "dept": "Engineering", "salary": 80000}
]

# Filtrar
engineers = [e for e in employees if e["dept"] == "Engineering"]

# Ordenar
by_salary = sorted(employees, key=lambda e: e["salary"], reverse=True)

# Extraer campo
names = [e["name"] for e in employees]

# Agrupar
from collections import defaultdict
by_dept = defaultdict(list)
for e in employees:
    by_dept[e["dept"]].append(e)

# Calcular
total_salary = sum(e["salary"] for e in employees)
avg_salary = total_salary / len(employees)
```

---

7. Comprensiones

List Comprehension

```python
# Sintaxis: [expresión for item in iterable if condición]

# Básica
squares = [x**2 for x in range(10)]
```

```python
# Con condición
evens = [x for x in range(20) if x % 2 == 0]

# Con transformación condicional
signs = ["+" if x > 0 else "-" if x < 0 else "0" for x in numbers]

# Múltiples for
pairs = [(x, y) for x in range(3) for y in range(3)]
# Equivale a:
# for x in range(3):
#     for y in range(3):
#         pairs.append((x, y))

# Con condición en for anidado
pairs = [(x, y) for x in range(3) for y in range(3) if x != y]

# Aplanar matriz
flat = [cell for row in matrix for cell in row]

# Con walrus operator
results = [y for x in data if (y := process(x)) is not None]
```

Dict Comprehension

```python
# Sintaxis: {key: value for item in iterable if condición}

# Básica
squares = {x: x**2 for x in range(5)}
# {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}

# Invertir diccionario
original = {"a": 1, "b": 2}
inverted = {v: k for k, v in original.items()}
# {1: "a", 2: "b"}

# Filtrar
filtered = {k: v for k, v in data.items() if v > 0}

# Transformar keys
upper_keys = {k.upper(): v for k, v in data.items()}

# Desde dos listas
keys = ["a", "b", "c"]
values = [1, 2, 3]
d = {k: v for k, v in zip(keys, values)}
# O simplemente: dict(zip(keys, values))
```

Set Comprehension

```python
# Sintaxis: {expresión for item in iterable if condición}

# Básica
unique_lengths = {len(word) for word in words}

# Con condición
```

```python
evens = {x for x in range(20) if x % 2 == 0}

# Eliminar duplicados con transformación
normalized = {s.lower().strip() for s in strings}
```

Generator Expression

```python
# Sintaxis: (expresión for item in iterable if condición)

# No crea lista en memoria, evalúa lazy
sum_squares = sum(x**2 for x in range(1000000))

# Útil para operaciones con grandes datos
any(x > 100 for x in huge_list)
all(x > 0 for x in numbers)

# Convertir a lista/tuple cuando sea necesario
list(x**2 for x in range(5))
tuple(x**2 for x in range(5))

# Encadenar generadores
pipeline = (process(x) for x in data)
filtered = (x for x in pipeline if x is not None)
```

---

8. Funciones Útiles

Funciones Built-in

```python
# Longitud
len([1, 2, 3])        # 3
len({"a": 1})         # 1
len("hello")          # 5

# Min/Max
min([3, 1, 4])        # 1
max([3, 1, 4])        # 4
min("hello")          # 'e' (orden ASCII)
max(users, key=lambda u: u["age"])  # Usuario más viejo

# Suma y estadísticas
sum([1, 2, 3])        # 6
sum([1, 2, 3], 10)    # 16 (con inicio)

# Ordenar
sorted([3, 1, 4])     # [1, 3, 4]
sorted([3, 1, 4], reverse=True)  # [4, 3, 1]
sorted(users, key=lambda u: u["name"])

# Reverso
list(reversed([1, 2, 3]))  # [3, 2, 1]

# Enumerate
for i, item in enumerate(["a", "b", "c"]):
    print(f"{i}: {item}")
```

```python
for i, item in enumerate(items, start=1):
    print(f"{i}: {item}")

# Zip
for a, b in zip([1, 2], ["a", "b"]):
    print(a, b)

# zip_longest (itertools)
from itertools import zip_longest
list(zip_longest([1, 2], [1, 2, 3], fillvalue=0))
# [(1, 1), (2, 2), (0, 3)]

# Map
list(map(str.upper, ["a", "b"]))  # ["A", "B"]
list(map(lambda x: x*2, [1, 2]))  # [2, 4]

# Filter
list(filter(lambda x: x > 0, [-1, 0, 1, 2]))  # [1, 2]
list(filter(None, [0, 1, "", "a"]))  # [1, "a"] (truthy)

# Any/All
any([False, True, False])   # True
all([True, True, False])    # False
any(x > 10 for x in nums)   # Con generador
```

itertools

```python
from itertools import (
    chain, combinations, permutations,
    groupby, islice, cycle, repeat,
    product, accumulate
)

# chain - concatenar iterables
list(chain([1, 2], [3, 4]))  # [1, 2, 3, 4]
list(chain.from_iterable([[1, 2], [3, 4]]))  # [1, 2, 3, 4]

# combinations - combinaciones sin repetición
list(combinations("ABC", 2))  # [('A','B'), ('A','C'), ('B','C')]

# permutations - permutaciones
list(permutations("AB"))  # [('A','B'), ('B','A')]

# product - producto cartesiano
list(product([1, 2], ["a", "b"]))
# [(1, 'a'), (1, 'b'), (2, 'a'), (2, 'b')]

# groupby - agrupar consecutivos
data = [("a", 1), ("a", 2), ("b", 3)]
for key, group in groupby(data, key=lambda x: x[0]):
    print(key, list(group))

# islice - slice para iteradores
list(islice(range(100), 5, 10))  # [5, 6, 7, 8, 9]
```

```python
# cycle - repetir infinitamente
for i, c in zip(range(6), cycle("ABC")):
    print(c)  # A B C A B C

# accumulate - sumas acumuladas
list(accumulate([1, 2, 3, 4]))  # [1, 3, 6, 10]
```

functools

```python
from functools import reduce, partial, lru_cache

# reduce - reducir a un valor
reduce(lambda a, b: a + b, [1, 2, 3, 4])  # 10
reduce(lambda a, b: a * b, [1, 2, 3, 4])  # 24

# partial - función con argumentos prefijados
def power(base, exp):
    return base ** exp

square = partial(power, exp=2)
cube = partial(power, exp=3)
square(5)  # 25

# lru_cache - memoización
@lru_cache(maxsize=128)
def fibonacci(n):
    if n < 2:
        return n
    return fibonacci(n-1) + fibonacci(n-2)
```

---

9. Patrones Comunes

Eliminar Duplicados

```python
# Simple (no preserva orden)
unique = list(set(items))

# Preservando orden
unique = list(dict.fromkeys(items))

# Con función personalizada
def unique_by(items, key):
    seen = set()
    result = []
    for item in items:
        k = key(item)
        if k not in seen:
            seen.add(k)
            result.append(item)
    return result
```

Agrupar Elementos

```
from collections import defaultdict

# Agrupar por atributo
def group_by(items, key):
    groups = defaultdict(list)
    for item in items:
        groups[key(item)].append(item)
    return dict(groups)

# Ejemplo
users = [
    {"name": "Alice", "dept": "Eng"},
    {"name": "Bob", "dept": "Sales"},
    {"name": "Charlie", "dept": "Eng"}
]
by_dept = group_by(users, lambda u: u["dept"])
```

Merge/Combine

```
# Merge listas alternando
def interleave(*iterables):
    from itertools import chain, zip_longest
    return [x for x in chain.from_iterable(
        zip_longest(*iterables, fillvalue=None)
    ) if x is not None]

# Merge diccionarios profundo
def deep_merge(d1, d2):
    result = d1.copy()
    for key, value in d2.items():
        if key in result and isinstance(result[key], dict) and isinstance(value, dict):
            result[key] = deep_merge(result[key], value)
        else:
            result[key] = value
    return result
```

Frecuencias y Top-N

```
from collections import Counter

# Top N elementos
def top_n(items, n):
    return Counter(items).most_common(n)

# Elemento más común
def most_common(items):
    return Counter(items).most_common(1)[0][0]

# Frecuencia como porcentaje
def frequencies(items):
    c = Counter(items)
    total = sum(c.values())
    return {k: v/total for k, v in c.items()}
```

10. Rendimiento (Big O)

Complejidad de Listas

Operación	Promedio	Peor
`list[i]`	O(1)	O(1)
`list[i] = x`	O(1)	O(1)
`list.append(x)`	O(1)	O(n)*
`list.pop()`	O(1)	O(1)
`list.pop(0)`	O(n)	O(n)
`list.insert(0, x)`	O(n)	O(n)
`x in list`	O(n)	O(n)
`list.remove(x)`	O(n)	O(n)
`len(list)`	O(1)	O(1)
`list.sort()`	O(n log n)	O(n log n)

*Amortizado O(1) por redimensionamiento dinámico

Complejidad de Diccionarios

Operación	Promedio	Peor
`dict[key]`	O(1)	O(n)
`dict[key] = value`	O(1)	O(n)
`key in dict`	O(1)	O(n)
`dict.get(key)`	O(1)	O(n)
`del dict[key]`	O(1)	O(n)
`len(dict)`	O(1)	O(1)
`dict.keys()`	O(1)	O(1)

*Peor caso es con muchas colisiones de hash

Complejidad de Sets

Operación	Promedio	Peor
`x in set`	O(1)	O(n)
`set.add(x)`	O(1)	O(n)
`set.remove(x)`	O(1)	O(n)
`set | set`	O(n+m)	O(n*m)
`set & set`	O(min(n,m))	O(n*m)
`set - set`	O(n)	O(n*m)

Recomendaciones

```python
#  Usar set para búsquedas frecuentes
valid_ids = set(get_valid_ids())  # O(1) lookup
if user_id in valid_ids:
    process(user_id)

#  Evitar búsqueda lineal en listas grandes
valid_ids = get_valid_ids()  # lista
```

```python
if user_id in valid_ids:     # O(n) cada vez!
    process(user_id)

#  Usar deque para operaciones en ambos extremos
from collections import deque
queue = deque()
queue.append(x)      # O(1) al final
queue.appendleft(x)  # O(1) al inicio
queue.popleft()      # O(1) desde inicio

#  list.pop(0) es O(n)
queue = []
queue.pop(0)         # O(n)

#  Usar dict para conteos
from collections import Counter
counts = Counter(items)  # O(n) total

#  Contar manualmente
counts = {}
for item in items:       # O(n)
    if item in counts:   # + O(1) cada vez
        counts[item] += 1
    else:
        counts[item] = 1
```

Recursos Relacionados

- **Anterior**: Python Basics Cheat Sheet
- **Siguiente**: OOP Python Cheat Sheet
- **Documentación**: docs.python.org/3/library/stdtypes.html

Cheat Sheet creado para el Bootcamp Python Zero to Hero - 2026